

Quiz 1: e-PGDiploma-DL&GenAI

Instructions:

- This quiz consists of **40 questions**.
- Questions may be **MCQ (single correct)** or **MSQ (multiple correct)**.
- For MSQ questions, **one or more options may be correct**.
- There is **no negative marking**.
- Select the **most appropriate answer(s)**.
- All questions carry equal marks.

Marking Scheme:

- **+1** for each correct answer.
- **0** for incorrect or partially correct answers.

Section A: Neural Network Fundamentals

1. A single neuron has inputs $x_1 = 2$, $x_2 = -1$ with weights $w_1 = 0.5$, $w_2 = 0.3$ and bias $b = 0.1$. If the activation function is ReLU, what is the output of this neuron?

- A. 0
- B. 0.8
- C. -0.8
- D. 0.1

2. The sigmoid function is $\sigma(z) = \frac{1}{1+e^{-z}}$. If $\sigma(z) = 0.8$, what is the value of $\sigma'(z)$?

- A. 0.16
- B. 0.20
- C. 0.64
- D. 0.80

3. A network's output layer produces logits $[2.0, 1.0, 0.0]$ for a 3-class problem. After applying softmax, what is the **approximate probability of class 0** (the first logit)?
- A. 0.67
 - B. 0.24
 - C. 0.09
 - D. 0.50
4. A fully connected neural network has an input layer of size 784, one hidden layer of size 128 (with bias), and an output layer of size 10 (with bias). How many **total trainable parameters** does this network have?
- A. 101,770
 - B. 101,632
 - C. 100,480
 - D. 101,898
5. For a binary classification task, the model predicts $\hat{y} = 0.9$ for a sample whose true label is $y = 1$. What is the binary cross-entropy loss for this single sample?
- A. 0.105
 - B. 0.9
 - C. 0.01
 - D. 2.303
6. A weight w currently has value 1.5 and its gradient $\frac{\partial L}{\partial w} = -0.4$. If the learning rate $\alpha = 0.1$, what is the updated value of w after one step of gradient descent?
- A. 1.54
 - B. 1.46
 - C. 1.10
 - D. 1.90
7. What mathematical principle does backpropagation use to compute gradients layer by layer?

- A. Fourier Transform
 - B. The chain rule of calculus
 - C. Matrix inversion
 - D. L'Hôpital's rule
8. Which of the following are valid reasons why ReLU is preferred over Sigmoid as an activation function in hidden layers of deep networks?
- A. ReLU helps mitigate the vanishing gradient problem because its gradient is 1 for positive inputs
 - B. ReLU is computationally cheaper since it only involves a thresholding operation
 - C. ReLU always outputs values between 0 and 1, making it easier to interpret
 - D. ReLU can lead to sparse activations, which can act as a form of regularization
9. Consider the backpropagation algorithm. Which of the following statements are **true**?
- A. Backpropagation computes gradients of the loss with respect to each weight using the chain rule
 - B. Backpropagation requires a differentiable loss function and differentiable activation functions
 - C. Backpropagation updates weights during the forward pass itself
 - D. Gradients computed during backpropagation can become extremely small in deep networks, a phenomenon known as the vanishing gradient problem
10. Batch Normalization is a widely used technique in deep learning. Which of the following correctly describe its effects?
- A. It normalizes the inputs to each layer to have approximately zero mean and unit variance
 - B. It introduces two learnable parameters (scale γ and shift β) per feature
 - C. It completely eliminates the need for dropout in all cases
 - D. It allows the use of higher learning rates by reducing internal covariate shift
11. Which of the following are effective strategies to address **overfitting** in neural networks?

- A. Applying dropout during training
- B. Increasing the number of layers and neurons in the network
- C. Using early stopping based on validation loss
- D. Applying L2 regularization (weight decay) to the loss function

12. The choice of loss function depends on the task. Which of the following pairings of task and loss function are **appropriate**?

- A. Multi-class classification (mutually exclusive) $\rightarrow$ Cross-Entropy Loss
- B. Binary classification $\rightarrow$ Binary Cross-Entropy (BCE) Loss
- C. Regression $\rightarrow$ Softmax Loss
- D. Multi-label classification $\rightarrow$ Binary Cross-Entropy applied per label

Section B: CNN Concepts and Computation

13. An input feature map of size 32×32 is passed through a convolutional layer with kernel size 5, stride 2, and padding 1. What is the spatial size of the output feature map?

- A. 14×14
- B. 15×15
- C. 16×16
- D. 28×28

14. A `nn.Conv2d` layer has 16 input channels, 32 output channels, and a kernel size of 3×3 with bias enabled. How many trainable parameters does this layer have?

- A. 4,640
- B. 4,608
- C. 4,096
- D. 14,400

15. Consider the following 4×4 feature map. After applying 2×2 max pooling with stride 2, what is the value at position $(0, 0)$ of the output?

$$\begin{bmatrix} 5 & 3 & 8 & 1 \\ 2 & 7 & 4 & 6 \\ 1 & 9 & 3 & 2 \\ 4 & 0 & 5 & 8 \end{bmatrix}$$

- A. 7
- B. 5
- C. 9
- D. 8

16. A CNN has two consecutive 3×3 convolutional layers, both with stride 1 and no padding. What is the effective receptive field size of a single neuron in the second layer's output with respect to the original input?

- A. 3×3
- B. 5×5
- C. 6×6
- D. 9×9

17. Which of the following are advantages of using convolutional layers over fully connected layers for processing image data?

- A. Parameter sharing—the same filter weights are applied across the entire input
- B. Translation equivariance—a shifted input produces a correspondingly shifted feature map
- C. Convolutional layers can capture global context in a single layer more effectively than fully connected layers
- D. Convolutional layers exploit local spatial structure by using small receptive fields

18. Regarding pooling layers in CNNs, which of the following statements are **correct**?

- A. Max pooling selects the maximum value in each pooling window, which helps retain dominant features
- B. Pooling layers have trainable parameters that are learned during backpropagation

- C. Pooling reduces the spatial dimensions of the feature map, thereby lowering computational cost
 - D. Average pooling computes the mean of values in each pooling window and can be useful in the final layers of a network
- 19.** A 1×1 convolution (pointwise convolution) is used in several modern CNN architectures. Which of the following are valid purposes of a 1×1 convolution?
- A. Reducing the number of channels (dimensionality reduction) before expensive 3×3 or 5×5 convolutions
 - B. Increasing the spatial resolution of the feature map
 - C. Adding non-linearity when followed by an activation function, without changing spatial dimensions
 - D. Learning cross-channel interactions while keeping spatial dimensions intact
- 20.** Transfer learning is commonly used in CNN-based tasks. Which of the following are **correct** practices when applying transfer learning?
- A. Freezing early convolutional layers (which learn generic low-level features) and fine-tuning later layers
 - B. Replacing the final classification head to match the number of classes in the new task
 - C. Always training the entire pre-trained network from scratch with a high learning rate
 - D. Using a smaller learning rate for pre-trained layers compared to newly added layers
- 21.** Data augmentation is a key technique for improving CNN generalization. Which of the following are commonly used data augmentation strategies for image classification tasks?
- A. Random horizontal flipping and random cropping
 - B. Randomly shuffling the pixel values within each image
 - C. Applying random rotation, color jitter, and scaling
 - D. Mixup—creating new training samples by linearly interpolating between two images and their labels
- 22.** What problem does the ResNet shortcut (identity) connection primarily solve?
- A. Overfitting on small datasets

- B. The degradation problem—deeper plain networks can perform worse than shallower ones even on training data
- C. High GPU memory cost of deep networks
- D. The inability of CNNs to process variable-resolution images

23. In a ResNet bottleneck block ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$), what is the role of the *first* 1×1 convolution?

- A. To increase spatial resolution by learned upsampling
- B. To reduce channel dimensionality before the expensive 3×3 convolution
- C. To apply global context across the entire feature map
- D. To replace batch normalisation in the block

24. What is the core innovation of the Inception module (GoogLeNet)?

- A. It replaces all convolutions with self-attention mechanisms
- B. It applies multiple kernel sizes (1×1 , 3×3 , 5×5) and pooling in parallel, then concatenates the results channel-wise
- C. It introduces residual skip connections between every two layers
- D. It uses transposed convolutions to upsample feature maps

25. MobileNetV2 uses **Inverted Residuals with Linear Bottlenecks**. How does the internal structure differ from a standard ResNet bottleneck block?

- A. ResNet compresses channels in the middle (wide $\rightarrow$ narrow $\rightarrow$ wide); MobileNetV2 expands in the middle (narrow $\rightarrow$ wide $\rightarrow$ narrow)—hence “inverted”
- B. MobileNetV2 uses depthwise separable convolutions in the expanded middle layer, reducing computation
- C. MobileNetV2 removes the non-linearity (ReLU) at the final 1×1 projection to prevent information loss at low dimensions
- D. All of the above

26. You are fine-tuning a ResNet on a medical dataset. The images are X-rays (grayscale). Which strategy is **LEAST** effective?

- A. Initializing with ImageNet weights and unfreezing all layers
- B. Using a very high learning rate on the frozen backbone
- C. Modifying the first conv layer to accept 1 channel instead of 3
- D. Using discriminative learning rates

27. If your CNN loss is not decreasing from the first epoch and gradients are zero, what is the most likely culprit?

- A. Too many layers
- B. A “Dead ReLU” problem where all neurons are in the negative trap
- C. Using Softmax instead of Sigmoid
- D. Using a Batch Size that is too large

Section C: Encoder-Decoder and Sequence Models

28. In an RNN, what does the hidden state primarily represent?

- A. Only current input
- B. Entire dataset statistics
- C. Information from previous time steps
- D. Output probabilities

29. Which of the following issues are commonly associated with vanilla RNNs?

- A. Vanishing gradients
- B. Exploding gradients
- C. Inability to model sequential data
- D. Difficulty in learning long-term dependencies

30. What is the key idea behind LSTM networks?

- A. Use convolution instead of recurrence
- B. Introduce gating mechanisms to control information flow

- C. Remove hidden states entirely
 - D. Use attention instead of recurrence
- 31.** Which components are present in an LSTM cell?
- A. Forget gate
 - B. Input gate
 - C. Output gate
 - D. Convolution gate
- 32.** In sequence-to-sequence (encoder-decoder) models, what is the role of the encoder?
- A. Generate output sequence
 - B. Compute loss function
 - C. Convert input sequence into a context representation
 - D. Apply softmax
- 33.** What is the main objective of an autoencoder?
- A. Classify input data
 - B. Predict future time steps
 - C. Reconstruct the input from a latent representation
 - D. Maximize classification accuracy
- 34.** Which of the following are true about autoencoders?
- A. They can learn non-linear representations
 - B. Without activation functions, they behave like PCA
 - C. They always require labeled data
 - D. They can be trained using reconstruction loss
- 35.** What is the purpose of sparsity constraint in sparse autoencoders?
- A. Increase number of parameters
 - B. Ensure all neurons are active
 - C. Encourage most neurons to remain inactive
 - D. Remove reconstruction loss

Section D: PyTorch and Training Dynamics

36. The following model is intended for classifying 28×28 grayscale images into 10 classes. Which line will cause a **runtime error** when a batch is passed through?

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, 3, padding=1) # Line A
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(3136, 10) # Line C

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = x.view(-1) # Line B
        x = F.log_softmax(self.fc1(x), dim=1) # Line D
        return x
```

- A. Line A: wrong input channels in Conv2d
- B. Line B: `x.view(-1)` collapses the batch dimension; should use `x.view(x.size(0), -1)`
- C. Line C: `nn.Linear` input size 3136 is incorrect
- D. Line D: should use `nn.CrossEntropyLoss()` instead of `log_softmax`

37. A student writes the following training loop. The loss keeps **increasing** over epochs. What is the most likely bug?

```
for epoch in range(10):
    for data, target in train_loader:
        output = model(data)
        loss = criterion(output, target)
        optimizer.step()
        optimizer.zero_grad()
        loss.backward()
```

- A. `loss.backward()` should come before `optimizer.zero_grad()`
- B. `optimizer.step()` is called before `loss.backward()`, so gradients are stale/zero
- C. The learning rate is too low
- D. `model.train()` is missing

38. You are building a CNN for RGB images of size 64×64 . After passing through the layers below with batch size 8, what is the final tensor shape?

```

self.conv1 = nn.Conv2d(3, 16, kernel_size=3, stride=1, padding=1)
self.conv2 = nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1)
self.pool = nn.MaxPool2d(2, 2)

# forward:
x = self.pool(F.relu(self.conv1(x))) # step 1
x = self.pool(F.relu(self.conv2(x))) # step 2

```

- A. [8, 32, 15, 15]
- B. [8, 32, 16, 16]
- C. [8, 32, 14, 14]
- D. [8, 16, 16, 16]

39. Consider the following optimizer and scheduler setup. After completing 3 full epochs of training, what will be the approximate learning rate?

```

optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
scheduler = torch.optim.lr_scheduler.StepLR(
    optimizer, step_size=2, gamma=0.1
)

for epoch in range(3):
    train_one_epoch(model, train_loader, optimizer)
    scheduler.step()

```

- A. 0.001
- B. 0.0001
- C. 0.00001
- D. 0.0007

40. Which statement about `model.eval()` in PyTorch is most accurate?

- A. It disables gradient computation globally
- B. It switches Dropout and BatchNorm to inference behaviour, but gradients are still tracked unless `torch.no_grad()` is also used
- C. It freezes all parameters permanently
- D. It automatically moves the model to CUDA